

10. Data Types

The JDBC driver converts the Java data type to the appropriate JDBC type, before sending it to the database. It uses a default mapping for most data types. For example, a Java int is converted to an SQL INTEGER. Default mappings were created to provide consistency between drivers.

The following table summarizes the default JDBC data type that the Java data type is converted to, when you call the setXXX() method of the PreparedStatement or CallableStatement object or the ResultSet.updateXXX() method.

10.1 SQL vs Java Data Types

SQL	JDBC/Java	setXXX	updateXXX
VARCHAR	java.lang.String	setString	updateString
CHAR	java.lang.String	setString	updateString
LONGVARCHAR	java.lang.String	setString	updateString
BIT	boolean	setBoolean	updateBoolean
NUMERIC	java.math.BigDecimal	setBigDecimal	updateBigDecimal
TINYINT	byte	setByte	updateByte
SMALLINT	short	setShort	updateShort
INTEGER	int	setInt	updateInt
BIGINT	long	setLong	updateLong
REAL	float	setFloat	updateFloat
FLOAT	float	setFloat	updateFloat
DOUBLE	double	setDouble	updateDouble
VARBINARY	byte[]	setBytes	updateBytes
BINARY	byte[]	setBytes	updateBytes
DATE	java.sql.Date	setDate	updateDate
TIME	java.sql.Time	setTime	updateTime

TIMESTAMP	java.sql.Timestamp	setTimestamp	updateTimestamp
CLOB	java.sql.Clob	setClob	updateClob
BLOB	java.sql.Blob	setBlob	updateBlob
ARRAY	java.sql.Array	setARRAY	updateARRAY
REF	java.sql.Ref	SetRef	updateRef
STRUCT	java.sql.Struct	SetStruct	updateStruct
XML	SQLXML	setSQLXML	updateSQLXML
ROWID	RowId	setRowId	updateRowId
DISTINCT	BigDecimal	setBigDecimal	updateBigDecimal
DATALINK	Java.netURL	setURL	updateURL
NCLOB	NClob	setNClob	updateNClob

JDBC 3.0 has enhanced support for BLOB, CLOB, ARRAY, and REF data types. The `ResultSet` object now has `updateBLOB()`, `updateCLOB()`, `updateArray()`, and `updateRef()` methods that enable you to directly manipulate the respective data on the server.

The `setXXX()` and `updateXXX()` methods enable you to convert specific Java types to specific JDBC data types. The methods, `setObject()` and `updateObject()`, enable you to map almost any Java type to a JDBC data type.

`ResultSet` object provides corresponding `getXXX()` method for each data type to retrieve column value. Each method can be used with column name or by its ordinal position.

10.2 ResultSet Methods to Get/Set SQL Values

SQL	JDBC/Java	setXXX	getXXX
VARCHAR	java.lang.String	setString	getString
CHAR	java.lang.String	setString	getString
LONGVARCHAR	java.lang.String	setString	getString
BIT	boolean	setBoolean	getBoolean
NUMERIC	java.math.BigDecimal	setBigDecimal	getBigDecimal

TINYINT	byte	setByte	getByte
SMALLINT	short	setShort	getShort
INTEGER	int	setInt	getInt
BIGINT	long	setLong	getLong
REAL	float	setFloat	getFloat
FLOAT	float	setFloat	getFloat
DOUBLE	double	setDouble	getDouble
VARBINARY	byte[]	setBytes	getBytes
BINARY	byte[]	setBytes	getBytes
DATE	java.sql.Date	setDate	getDate
TIME	java.sql.Time	setTime	getTime
TIMESTAMP	java.sql.Timestamp	setTimestamp	getTimestamp
CLOB	java.sql.Clob	setClob	getClob
BLOB	java.sql.Blob	setBlob	getBlob
ARRAY	java.sql.Array	setARRAY	getARRAY
REF	java.sql.Ref	SetRef	getRef
STRUCT	java.sql.Struct	SetStruct	getStruct
XML	SQLXML	setSQLXML	getSQLXML
ROWID	RowId	setRowId	getRowId
DISTINCT	BigDecimal	setBigDecimal	getBigDecimal
DATALINK	Java.net.URL	setURL	getURL
NCLOB	NClob	setNClob	getNClob

10.3 Date & Time Data Types

The `java.sql.Date` class maps to the SQL `DATE` type, and the `java.sql.Time` and `java.sql.Timestamp` classes map to the SQL `TIME` and SQL `TIMESTAMP` data types, respectively.

```

package com.lokesh;

public class DataAndTimeExample1 {
    public static void main(String[] args) {
        java.util.Date javaDate = new java.util.Date();
        long javaTime = javaDate.getTime();
        System.out.println("The Java Date is: " + javaDate.toString());

        // Get and display SQL DATE
        java.sql.Date sqlDate = new java.sql.Date(javaTime);
        System.out.println("The SQL DATE is: " + sqlDate.toString());

        // Get and display SQL TIME
        java.sql.Time sqlTime = new java.sql.Time(javaTime);
        System.out.println("The SQL TIME is: " + sqlTime.toString());
        // Get and display SQL TIMESTAMP
        java.sql.Timestamp sqlTimestamp = new java.sql.Timestamp(javaTime);
        System.out.println("The SQL TIMESTAMP is: " + sqlTimestamp.toString());
    }
}

```

Output:

```

The Java Date is: Wed Dec 04 12:26:39 IST 2024
The SQL DATE is: 2024-12-04
The SQL TIME is: 12:26:39
The SQL TIMESTAMP is: 2024-12-04 12:26:39.916

```

SQL Query to create table for Example2

CREATE DATABASE IF NOT EXISTS organization;

use organization;

```

CREATE TABLE IF NOT EXISTS DateTimeExample (
    id INT AUTO_INCREMENT PRIMARY KEY,
    event_date DATE NOT NULL,
    event_time TIME NOT NULL,
    event_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```

package com.lokesh;

import java.sql.*;

public class DateAndTimeExample2 {
    public static void main(String[] args) {
        // JDBC URL, user name, and password
        String jdbcUrl = "jdbc:mysql://localhost:3306/organization";
        String username = "root";
        String password = "root";

        Connection connection = null;
        PreparedStatement preparedStatement = null;
        Statement statement = null;
        ResultSet resultSet = null;

        try {
            // Establish connection
            connection = DriverManager.getConnection(jdbcUrl, username, password);

            // Insert data

```

```

String insertQuery = "INSERT INTO DateTimeExample (event_date, event_time,
                                                    event_timestamp) VALUES (?, ?, ?)";
preparedStatement = connection.prepareStatement(insertQuery);

java.sql.Date sqlDate = java.sql.Date.valueOf("2024-12-04"); // YYYY-MM-DD
java.sql.Time sqlTime = java.sql.Time.valueOf("14:30:00"); // HH:MM:SS
java.sql.Timestamp sqlTimestamp = java.sql.Timestamp.valueOf("2024-12-04
                                                             14:30:00"); // YYYY-MM-DD HH:MM:SS

preparedStatement.setDate(1, sqlDate);
preparedStatement.setTime(2, sqlTime);
preparedStatement.setTimestamp(3, sqlTimestamp);

int rowsInserted = preparedStatement.executeUpdate();
System.out.println("Rows inserted: " + rowsInserted);

// Retrieve data
String selectQuery = "SELECT id, event_date, event_time, event_timestamp
                     FROM DateTimeExample";

statement = connection.createStatement();
resultSet = statement.executeQuery(selectQuery);

System.out.println("Retrieved Data:");
System.out.printf("%-5s %-12s %-12s %-20s\n", "ID", "Date", "Time",
                                                "Timestamp");
System.out.println("-----");

while (resultSet.next()) {
    int id = resultSet.getInt("id");
    Date eventDate = resultSet.getDate("event_date");
    Time eventTime = resultSet.getTime("event_time");
    Timestamp eventTimestamp = resultSet.getTimestamp("event_timestamp");

    System.out.printf("%-5d %-12s %-12s %-20s\n", id, eventDate,
                                                eventTime, eventTimestamp);
}

} catch (SQLException e) {
    e.printStackTrace();
} finally {
    // Close resources manually
    try {
        if (resultSet != null)
            resultSet.close();
        if (statement != null)
            statement.close();
        if (preparedStatement != null)
            preparedStatement.close();
        if (connection != null)
            connection.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

Output:

```

Rows inserted: 1
Retrieved Data:
ID      Date       Time       Timestamp
-----
1       2024-12-04    14:30:00   2024-12-04 14:30:00.0

```